Robot Handle Control

Robot Handle Control

1. Course Content
2. Preparation
3. Button Control
4. Node Communication
5. Source Code Analysis

1. Course Content

1. After running the program, use the PS2 handle to control the robot chassis and control the robotic arm's movement.
2. Master the key operations of handle control

2. Preparation

- Insert the wireless handle's USB receiver into the mainboard's USB port
- Start the handle control function

```
ros2 launch yahboomcar_bringup joy_contoller_launch.py
```

```
[joy_node-2] [INFO] [1782119193.643306032] [joy_node]: Opened joystick: Controller.  deadzone: 0.050000
[joy_control_node-3] [INFO] [1782119193.945162666] [joy_control]: Joy Control Node initialized
[joy_control_node-3] [INFO] [1782119193.945852126] [joy_control]: === Configuration Parameters ===
[joy_control_node-3] [INFO] [1782119193.946400845] [joy_control]: Deadzone: 0.1
[joy_control_node-3] [INFO] [1782119193.946908668] [joy_control]: Base command rate: 20.0 Hz
[joy_control_node-3] [INFO] [1782119193.947439883] [joy_control]: Arm command rate: 5.0 Hz
[joy_control_node-3] [INFO] [1782119193.947949369] [joy_control]: Max linear speed: 1.0 m/s
[joy_control_node-3] [INFO] [1782119193.948446055] [joy_control]: Max angular speed: 2.0 rad/s
[joy_control_node-3] [INFO] [1782119193.948942997] [joy_control]: Enable button: 11 (start)
[joy_control_node-3] [INFO] [1782119193.949455172] [joy_control]: Linear speed step: 0.05 m/s
[joy_control_node-3] [INFO] [1782119193.949964210] [joy_control]: Angular speed step: 0.1 rad/s
[joy_control_node-3] [INFO] [1782119193.950474049] [joy_control]: Axis mapping: linear_x=1, linear_y=2, angular_z=0
[joy_control_node-3] [INFO] [1782119193.950982671] [joy_control]: Arm joints: axis1=6, axis2=7
[joy_control_node-3] [INFO] [1782119193.951525311] [joy_control]: Button mapping: joint3_inc=1, joint3_dec=3, joint4_inc=4, joint4_dec=0
[joy_control_node-3] [INFO] [1782119193.952060078] [joy_control]: Speed toggle buttons: linear=13, angular=14
[joy_control_node-3] [INFO] [1782119193.952571772] [joy_control]: Arm mode button: 9
[joy_control_node-3] [INFO] [1782119193.953078219] [joy_control]: ==============================
[joy_control_node-3] [INFO] [1782119193.953571801] [joy_control]: Waiting for joystick input...
[joy_control_node-3] [INFO] [1782119193.995619971] [joy_control]: Joystick connected: 15 buttons, 8 axes
[joy_control_node-3] [INFO] [1782119193.996192243] [joy_control]: Button indices: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
[joy_control_node-3] [INFO] [1782119193.996935752] [joy_control]: Axis indices: [0, 1, 2, 3, 4, 5, 6, 7]
[joy_control_node-3] [INFO] [1782119193.997522745] [joy_control]: Initial enable button 11 state: 0
[joy_control_node-3] [INFO] [1782119193.998096553] [joy_control]: Initial arm mode button 9 state: 0
```

3. Button Control

- After the program starts, the handle is disabled by default and in a safe state; moving the joystick and pressing buttons will not actually send commands. Press **START** to enable it, and you can then begin control, with a buzzer prompt tone (beep—beep—beep)

```
[joy_control_node-3] [INFO] [1782123270.890294014] [joy_control]: Joystick ENABLED
```

> **Tip**

> Handle unlock (enable) prompt tone: beep—beep—beep
>
> Handle lock (disable) prompt tone: beep————

- Key operation table

Handle Action	Function
Left stick up/down	Chassis x-axis forward/backward
Left stick left/right	Chassis z-axis rotate left/right
Right stick left/right	Chassis translate left/right
"START" button	Handle enable/disable
Left button 1	Adjust X, Y-axis linear velocity
Left button 2	Adjust Z-axis angular velocity
D-pad left/right	Increase/decrease servo 1 angle (increase/decrease servo 5 angle)
D-pad up/down	Increase/decrease servo 2 angle (increase/decrease servo 5 angle)
Button X/Button B	Increase/decrease servo 3 angle
Button Y/Button A	Increase/decrease servo 4 angle
Right button 2	Switch the joint controlled by D-pad left/right and up/down (robotic arm joint 1/2, robotic arm joint 5/6)
Right button 1	Reset the robotic arm to the initial position

- Key layout diagram

START: Unlock/Lock Handle

Arrow keys
Up/Down: Gripper closed/open | Joint 2 up/down
Left/Right: Joint 5 increases/decreases | Joint 1 moves left/moves right

Y: Joint 4 up
A: Joint 4 down
X: Joint 3 down
B: Joint 3 up



Left rocker
Up/Down: forward/back
Left/Right: turn left/turn right

Right rocker
Left/Right: left/right translation
(Note: Only M1 PRO has left and right translation function)

R1: Robotic arm posture reset

L1: Line speed switching



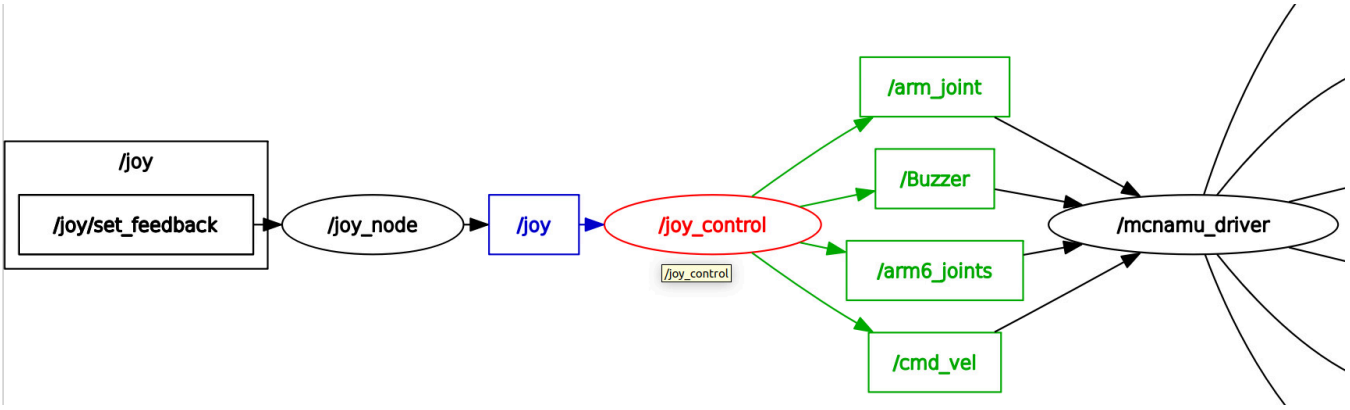
R2: Direction control switching
Joint 1, 2/Joint 5, 6

Status indicator light
Keep on indicates the handle and receiver, Pairing connection successful

L2: Angular velocity switching

4. Node Communication

```
rqt_graph
```



5. Source Code Analysis

- Source code path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/control/joy_control.py
```

```
    def joy_callback(self, msg):
        """Process joystick messages"""
        with self.lock:
            # On first message, print joystick info and initialize button state
            if not hasattr(self, 'joy_info_printed'):
                self.joy_info_printed = True
                self.get_logger().info(f"Joystick connected: {len(msg.buttons)} buttons,
{len(msg.axes)} axes")
                self.get_logger().info(f"Button indices: {list(range(len(msg.buttons)))}")
                self.get_logger().info(f"Axis indices: {list(range(len(msg.axes)))}")
                # Initialize last button states for edge detection
                if len(msg.buttons) > self.enable_button:
                    self.last_enable_button_state = msg.buttons[self.enable_button]
                    self.get_logger().info(f"Initial enable button {self.enable_button}
state: {self.last_enable_button_state}")
                if len(msg.buttons) > self.arm_mode_button:
                    self.last_arm_mode_button_state = msg.buttons[self.arm_mode_button]
                    self.get_logger().info(f"Initial arm mode button {self.arm_mode_button}
state: {self.last_arm_mode_button_state}")
                if len(msg.buttons) > self.button_reset_arm:
                    self.last_reset_arm_button_state = msg.buttons[self.button_reset_arm]
                    self.get_logger().info(f"Initial reset arm button
{self.button_reset_arm} state: {self.last_reset_arm_button_state}")
```

```
            # Update current axes for base control (always update, even when disabled)
            required_axes_count = max(self.axis_linear_x, self.axis_linear_y,
self.axis_angular_z) + 1
            if len(msg.axes) >= required_axes_count:
                self.current_axes = {
                    'linear_x': msg.axes[self.axis_linear_x],
                    'linear_y': msg.axes[self.axis_linear_y],
                    'angular_z': msg.axes[self.axis_angular_z]
                }
                self.get_logger().debug(f"Updated current_axes: {self.current_axes}")
            else:
                self.get_logger().warn(f"Not enough axes ({len(msg.axes)} <
{required_axes_count}), using zeros")
                self.current_axes = {'linear_x': 0.0, 'linear_y': 0.0, 'angular_z': 0.0}

            if len(msg.buttons) > self.enable_button:
                button_state = msg.buttons[self.enable_button]
                self.get_logger().debug(f"Enable button {self.enable_button} state:
{button_state}, last_state: {self.last_enable_button_state}")

                # Only detect rising edge (0 -> 1): toggle enable/disable on button press
                if button_state == 1 and self.last_enable_button_state == 0:
                    # Toggle the enabled state
                    self.enabled = not self.enabled
                    if self.enabled:
                        self.get_logger().info(f"Joystick ENABLED")
                    else:
                        self.get_logger().info(f"Joystick DISABLED")

                    # Play buzzer pattern as toggle feedback
                    if self.enabled:
                        pattern = [(0.1, 0.08), (0.1, 0.08), (0.1, 0.0)]
                    else:
                        pattern = [(0.9, 0.0)]
                    threading.Thread(
                        target=self._play_buzzer_pattern,
                        args=(pattern,),
                        daemon=True
                    ).start()

                # Update last state (only after processing)
                self.last_enable_button_state = button_state
            else:
                self.get_logger().warn(f"Button index {self.enable_button} out of range
(buttons count: {len(msg.buttons)})")

            if not self.enabled:
                return

            # Process arm mode toggle button (button 9) with edge detection
            if len(msg.buttons) > self.arm_mode_button:
                arm_mode_button_state = msg.buttons[self.arm_mode_button]
```

```
                # Only toggle on rising edge (0 -> 1)
                if arm_mode_button_state == 1 and self.last_arm_mode_button_state == 0:
                    self.arm_mode = 1 - self.arm_mode  # Toggle between 0 and 1
                    mode_str = "joint1/2" if self.arm_mode == 0 else "joint5/6"
                    self.get_logger().info(f"Arm control mode switched to: {mode_str}")

                # Update last state
                self.last_arm_mode_button_state = arm_mode_button_state

            # Process arm reset button (button 7) with edge detection
            if len(msg.buttons) > self.button_reset_arm:
                reset_button_state = msg.buttons[self.button_reset_arm]

                # Only trigger on rising edge (0 -> 1)
                if reset_button_state == 1 and self.last_reset_arm_button_state == 0:
                    # Reset joint positions to initial pose
                    init_pose_raw = self.get_parameter('init_pose').value
                    self.joint_positions = [int(x) for x in init_pose_raw]
                    # Publish reset command via ArmJoints message
                    self._pubSix_Arm(self.joint_positions, runtime=2000)
                    time.sleep(2.0)
                    self.get_logger().info(f"Arm reset to initial pose:
{self.joint_positions}")
                # Update last state
                self.last_reset_arm_button_state = reset_button_state

            # Process linear speed toggle button
            if len(msg.buttons) > self.button_linear_speed_toggle:
                if msg.buttons[self.button_linear_speed_toggle] == 1 and not
self.speed_toggle_buttons['linear']:
                    self.speed_toggle_buttons['linear'] = True
                    self.current_linear_speed += self.linear_speed_step
                    if self.current_linear_speed > self.max_linear_speed:
                        self.current_linear_speed = self.linear_speed_step
                    self.get_logger().info(f"Linear speed set to:
{self.current_linear_speed:.2f} m/s")
                elif msg.buttons[self.button_linear_speed_toggle] == 0:
                    self.speed_toggle_buttons['linear'] = False

            # Process angular speed toggle button
            if len(msg.buttons) > self.button_angular_speed_toggle:
                if msg.buttons[self.button_angular_speed_toggle] == 1 and not
self.speed_toggle_buttons['angular']:
                    self.speed_toggle_buttons['angular'] = True
                    self.current_angular_speed += self.angular_speed_step
                    if self.current_angular_speed > self.max_angular_speed:
                        self.current_angular_speed = self.angular_speed_step
                    self.get_logger().info(f"Angular speed set to:
{self.current_angular_speed:.2f} rad/s")
                elif msg.buttons[self.button_angular_speed_toggle] == 0:
                    self.speed_toggle_buttons['angular'] = False
```

```
            # Process joint control buttons (continuous movement while held, like axes)
            # Joint 3 increment (button 1)
            if len(msg.buttons) > self.button_joint3_inc:
                self.joint_buttons['joint3_inc'] = (msg.buttons[self.button_joint3_inc] ==
1)
                if self.joint_buttons['joint3_inc']:
                    self.joint_positions[2] = min(self.arm_bounds[2][1],
self.joint_positions[2] + self.arm_joint_step * 0.1)

            # Joint 3 decrement (button 3)
            if len(msg.buttons) > self.button_joint3_dec:
                self.joint_buttons['joint3_dec'] = (msg.buttons[self.button_joint3_dec] ==
1)
                if self.joint_buttons['joint3_dec']:
                    self.joint_positions[2] = max(self.arm_bounds[2][0],
self.joint_positions[2] - self.arm_joint_step * 0.1)

            # Joint 4 increment (button 4)
            if len(msg.buttons) > self.button_joint4_inc:
                self.joint_buttons['joint4_inc'] = (msg.buttons[self.button_joint4_inc] ==
1)
                if self.joint_buttons['joint4_inc']:
                    self.joint_positions[3] = min(self.arm_bounds[3][1],
self.joint_positions[3] + self.arm_joint_step * 0.1)

            # Joint 4 decrement (button 0)
            if len(msg.buttons) > self.button_joint4_dec:
                self.joint_buttons['joint4_dec'] = (msg.buttons[self.button_joint4_dec] ==
1)
                if self.joint_buttons['joint4_dec']:
                    self.joint_positions[3] = max(self.arm_bounds[3][0],
self.joint_positions[3] - self.arm_joint_step * 0.1)

            # Process axis-controlled joints (joint1/5 and joint2/6)
            if len(msg.axes) > max(self.axis_joint1, self.axis_joint2):
                # Apply deadzone and scaling
                axis1_value = msg.axes[self.axis_joint1]
                axis2_value = msg.axes[self.axis_joint2]

                # Apply deadzone
                if abs(axis1_value) < self.deadzone:
                    axis1_value = 0.0
                if abs(axis2_value) < self.deadzone:
                    axis2_value = 0.0

                # Scale to joint movement (assuming axis range -1 to 1)
                joint1_delta = axis1_value * self.arm_joint_step * 0.1  # Scale factor
                joint2_delta = axis2_value * self.arm_joint_step * 0.1

                if self.arm_mode == 0:
                    # Mode 0: axis6 -> joint1, axis7 -> joint2
                    self.joint_positions[0] = max(self.arm_bounds[0][0],
min(self.arm_bounds[0][1], self.joint_positions[0] + joint1_delta))
```

```
                    self.joint_positions[1] = max(self.arm_bounds[1][0],
min(self.arm_bounds[1][1], self.joint_positions[1] + joint2_delta))
                else:
                    # Mode 1: axis6 -> joint5, axis7 -> joint6
                    self.joint_positions[4] = max(self.arm_bounds[4][0],
min(self.arm_bounds[4][1], self.joint_positions[4] + joint1_delta))
                    self.joint_positions[5] = max(self.arm_bounds[5][0],
min(self.arm_bounds[5][1], self.joint_positions[5] + joint2_delta))
```